MySQL Database Schema for Friend Finder Application

1. Create Users Table

```
CREATE TABLE users (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  otp VARCHAR(255),
  is_verified BOOLEAN NOT NULL DEFAULT FALSE,
  profile_picture LONGTEXT,
  last_login DATETIME,
  activity_score INT NOT NULL DEFAULT 0,
  report_count INT NOT NULL DEFAULT 0
);
```

2. Create Interests Table

```
CREATE TABLE interests (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL UNIQUE
);
```

3. Create Friend Requests Table

```
CREATE TABLE friend_requests (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  sender_id BIGINT NOT NULL,
  receiver_id BIGINT NOT NULL,
  status VARCHAR(50) NOT NULL, -- e.g., 'PENDING', 'ACCEPTED', 'REJECTED'
```

```
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (sender_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (receiver_id) REFERENCES users(id) ON DELETE CASCADE
);
```

4. Create Messages Table

```
CREATE TABLE messages (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  sender_id BIGINT NOT NULL,
  receiver_id BIGINT NOT NULL,
  content LONGTEXT NOT NULL,
  timestamp DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (sender_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (receiver_id) REFERENCES users(id) ON DELETE CASCADE
);
```

5. Create Join Table: User Interests (Many-to-Many)

```
CREATE TABLE user_interests (
  user_id BIGINT NOT NULL,
  interest_id BIGINT NOT NULL,
  PRIMARY KEY (user_id, interest_id),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (interest_id) REFERENCES interests(id) ON DELETE CASCADE
);
```

6. Create Join Table: User Friends (Many-to-Many)

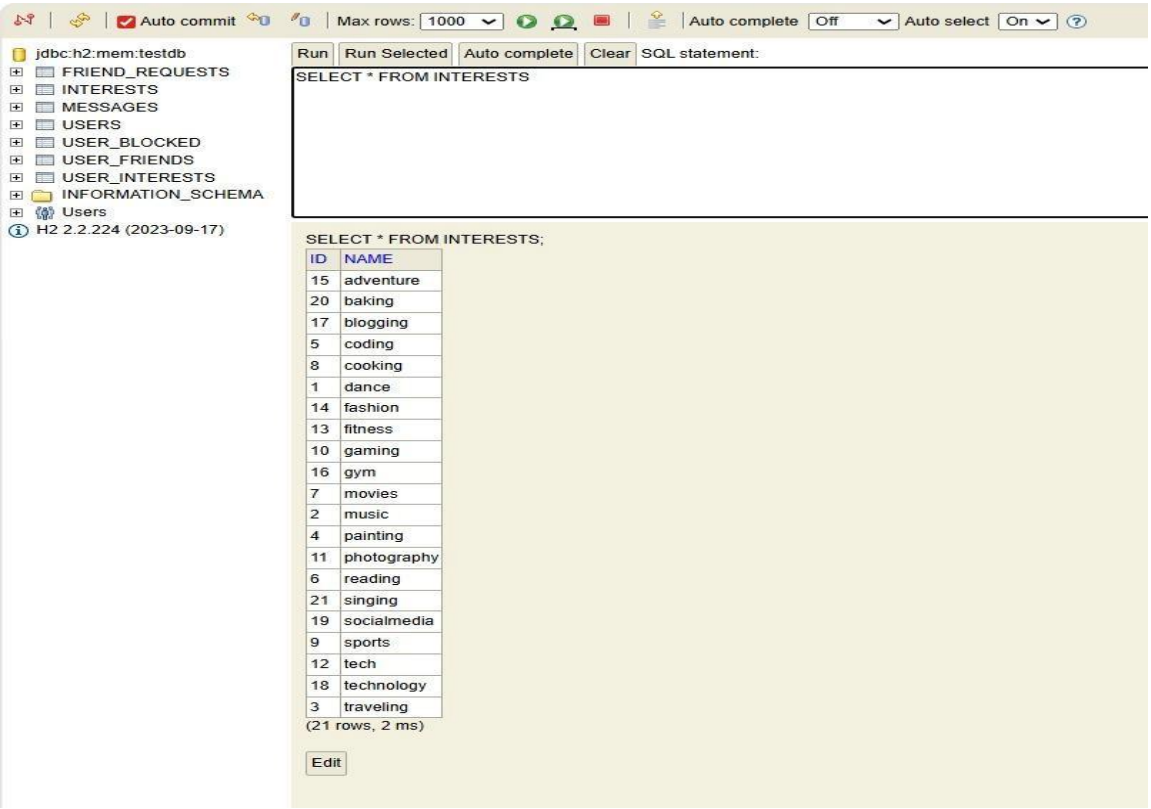
```
CREATE TABLE user_friends (
  user_id BIGINT NOT NULL,
```

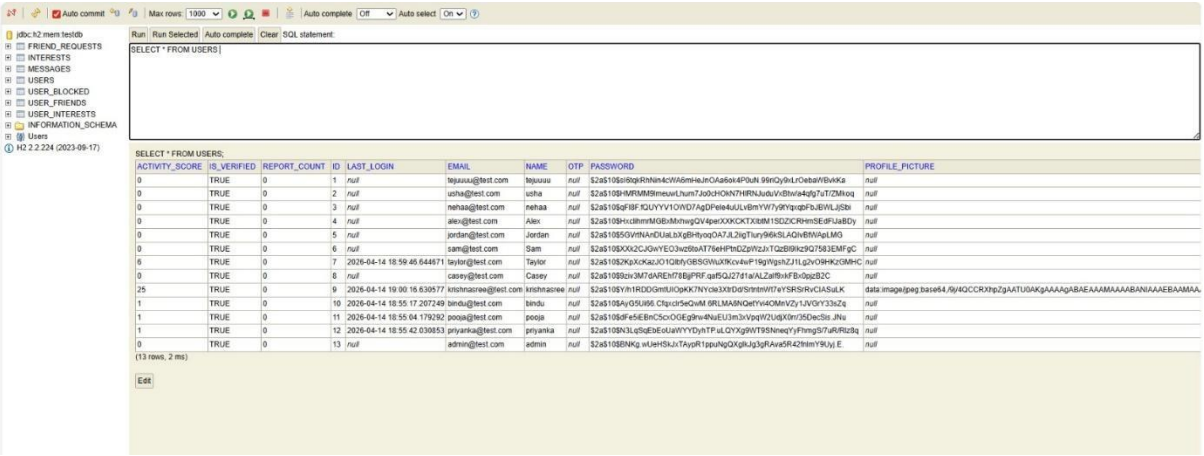
```
  friend_id BIGINT NOT NULL,
  PRIMARY KEY (user_id, friend_id),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (friend_id) REFERENCES users(id) ON DELETE CASCADE
);
```

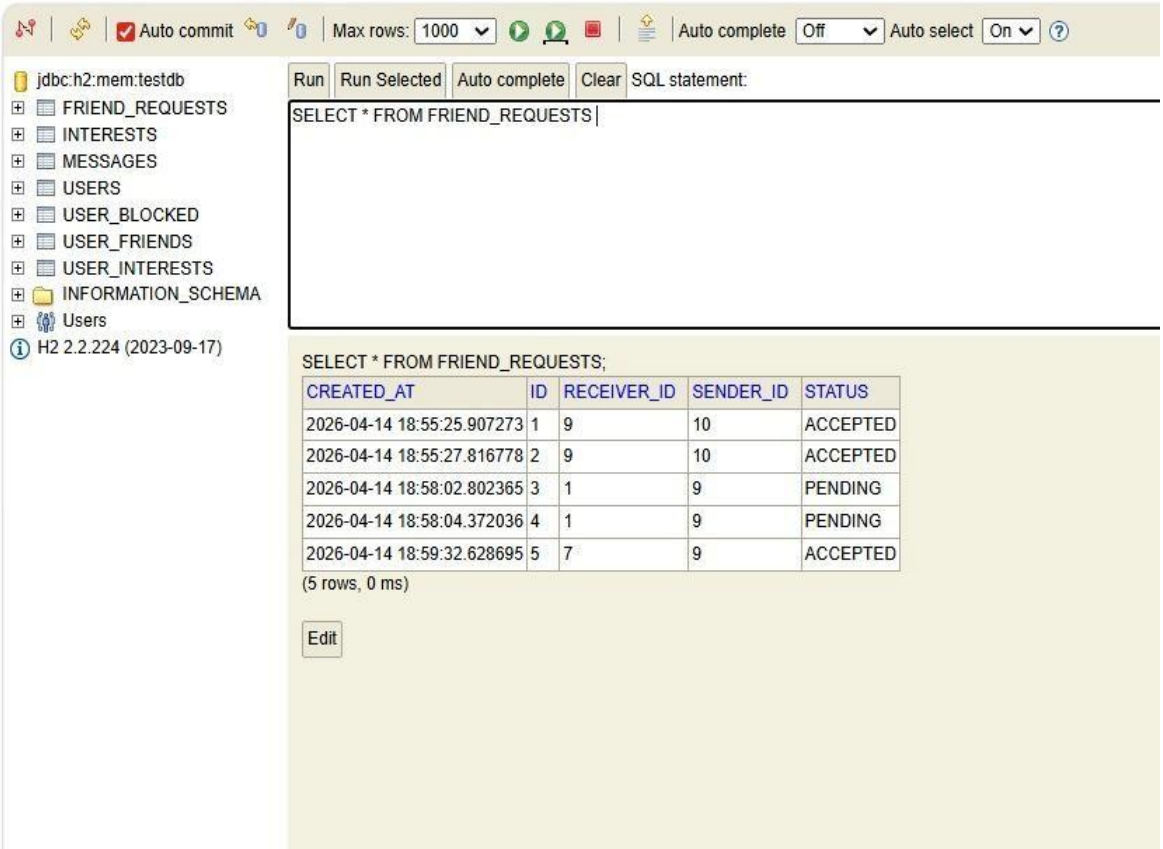
7. Create Join Table: User Blocked (Many-to-Many)

```
CREATE TABLE user_blocked (
  user_id BIGINT NOT NULL,
  blocked_id BIGINT NOT NULL,
  PRIMARY KEY (user_id, blocked_id),
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (blocked_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Screenshots of SQL Database :







CREATED_AT	ID	RECEIVER_ID	SENDER_ID	STATUS
2026-04-14 18:55:25.907273	1	9	10	ACCEPTED
2026-04-14 18:55:27.816778	2	9	10	ACCEPTED
2026-04-14 18:58:02.802365	3	1	9	PENDING
2026-04-14 18:58:04.372036	4	1	9	PENDING
2026-04-14 18:59:32.628695	5	7	9	ACCEPTED

Auto commit | Max rows: 1000 | Auto complete Off | Auto select On

jdbc:h2:mem:testdb
- FRIEND_REQUESTS
- INTERESTS
- MESSAGES
- USERS
- USER_BLOCKED
- USER_FRIENDS
- USER_INTERESTS
- INFORMATION_SCHEMA
- Users

H2 2.2.224 (2023-09-17)

Run | Run Selected | Auto complete | Clear | SQL statement:

```
SELECT * FROM USER_FRIENDS
```

SELECT * FROM USER_FRIENDS;

FRIEND_ID	USER_ID
9	10
10	9

(2 rows, 0 ms)

Edit

Auto commit | Max rows: 1000 | Auto complete Off | Auto select On

jdbc:h2:mem:testdb
- FRIEND_REQUESTS
- INTERESTS
- MESSAGES
- USERS
- USER_BLOCKED
- USER_FRIENDS
- USER_INTERESTS
- INFORMATION_SCHEMA
- Users

H2 2.2.224 (2023-09-17)

Run | Run Selected | Auto complete | Clear | SQL statement:

```
SELECT * FROM MESSAGES
```

SELECT * FROM MESSAGES;

ID	RECEIVER_ID	SENDER_ID	TIMESTAMP	CONTENT
1	10	9	2026-04-14 18:56:53.344888	hlo
2	10	9	2026-04-14 18:57:00.032668	Nice to meet you
3	9	7	2026-04-14 19:00:03.378833	hloo

(3 rows, 2 ms)

Edit